

Five end-to-end recipes that cover ~95% of integration cases — from new-patient onboarding to webhook-driven status sync. Each flow is multi-call: copy the steps in order, swap in your real token, and run them against `https://demo.prescribe-rx.com/api/v1`. Switch the language tab on each step to see cURL / PHP / JavaScript / Python equivalents.

Need a token? Open [the token matrix](#) to learn about token types, or grab a sandbox token from `GET https://demo.prescribe-rx.com/api/docs/tokens` to follow along live in the [playground](#).

FLOWS

1. [Catalog discovery](#) → [unified intake](#) → [status poll](#)
2. [Patient lookup](#) → [create encounter](#) → [prescribe](#)
3. [Bulk-orders for B2B clients](#)
4. [Webhook subscribe + signature verify](#)
5. [Test-data flow with is_sandbox](#)

1 Catalog discovery → unified intake → status poll

When you're: a sales org or client onboarding patients programmatically (the 1200-client onboarding use case)

Three calls. Discover what your tenant can sell, submit an intake, then poll for status until the encounter is prescribed. For real-time status, replace the poll loop with a webhook subscription (Flow 4).

Step 1 – pull catalog

Returns products, packages (with plans + items), all scoped to your token's tenant. Pricing fields are visibility-gated.

[cURL](#) [PHP](#) [JavaScript](#) [Python](#)

```
# Pulls products + packages scoped to your token's tenant.
curl -X GET "https://demo.prescribe-rx.com/api/v1/catalog" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json"
```

Step 2 – submit intake

One call creates the user (or matches by email), patient chart, encounter, intake answers, optional payment record, and links the selected product/package. Returns `encounter_id` + `encounter_number` for follow-up calls.

[cURL](#)

[PHP](#)

[JavaScript](#)

[Python](#)

```
curl -X POST "https://demo.prescribe-rx.com/api/v1/telehealth/intake/unified" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-H "Accept: application/json" \
-d '{
  "encounter_type_slug": "weight-management",
  "patient": {
    "first_name": "Jane",
    "last_name": "Smith",
    "email": "jane.smith@example.com",
    "date_of_birth": "1988-03-22",
    "phone": "555-555-9876",
    "shipping_address": {
      "street": "456 Oak Ave", "city": "Marietta",
      "state": "GA", "zip": "30060", "country": "US"
    },
    "billing_same_as_shipping": true
  },
  "vitals": { "height_inches": 64, "weight_lbs": 185 },
  "answers": { "current_weight_lbs": 185, "weight_goal": "lose_15_lbs" },
  "products": [
    { "product_type_slug": "tirzepatide", "quantity": 1 }
  ],
  "payment": {
    "mode": "reference_captured",
    "gateway": "your_gateway",
    "transaction": { "transaction_id": "ext-txn-123", "amount": 299.00 }
  },
  "consents": [
    { "consent_type": 6, "consent_text": "I consent to telehealth.", "signature": "" }
  ]
}'
```

Step 3 – poll status (or subscribe to webhook – see Flow 4)

Status transitions: `pending_intake` → `unassigned` → `pending_provider_review` → `provider_in_progress` → `prescribed` → `completed`. The poll endpoint also surfaces the linked order's payment + shipping status once the encounter is prescribed.

```
# Poll every 30-60 seconds until status reaches a terminal state
curl -X GET "https://demo.prescribe-rx.com/api/v1/telehealth/encounters/$ENCOUNTER_ID" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Accept: application/json"
```

2 Patient lookup → create encounter → prescribe

When you're a CLIENTPROVIDER who already has the patient on file and just needs to write a prescription

For client-provider tokens (token type 9). Skip the unified intake and use the encounter + prescription endpoints directly. The provider must have `prescription:sign` on the token to complete step 3.

Step 1 – find or create the patient

```
# Search by email (returns 0-N matches)
curl "https://demo.prescribe-rx.com/api/v1/patients/search?email=jane.smith" \
  -H "Authorization: Bearer $TOKEN" -H "Accept: application/json"

# If no match, create:
curl -X POST "https://demo.prescribe-rx.com/api/v1/patients" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{ "first_name": "Jane", "last_name": "Smith", "email": "...", "date_of_birth": "1980-01-01" }'
```

Step 2 – open an encounter

```
curl -X POST "https://demo.prescribe-rx.com/api/v1/encounters" \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "patient_chart_id": "$CHART_ID",
    "encounter_type_id": "$ENCOUNTER_TYPE_ID",
    "reason_for_visit": "GLP-1 follow-up"
  }'
```

Step 3 – prescribe + sign

POST .../prescribe validates the proposed Rx against the encounter's preclusions, then signs + locks it. Requires prescription:sign .

```
curl -X POST "https://demo.prescribe-rx.com/api/v1/encounters/$ENCOUNTER_ID" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "products": [
    { "product_id": "...", "dose_product_id": "...", "quantity": 1, "dire
  ],
  "compounding_reason_id": "..."
}'
```

GLP-1 prescriptions require a `compounding_reason_id` referencing the patient's documented medical-necessity rationale (controlled-substance + compliance enforcement). See `docs/dev/COMPOUNDING_REASONS_DEVELOPER_GUIDE.md` .

3 Bulk-orders for B2B clients

When you're: a clinic / wholesale buyer placing one big order against your client account, not creating individual encounters per patient

Bulk orders skip the encounter system entirely and ship products to the **client** (not patients). Used for clinic stock orders, wholesale shipments, and B2B sample programs. Asynchronous — the API returns a `request_id` immediately; poll `/bulk-orders/{requestId}/status` for processing state.

Step 1 – submit the bulk order

```
curl -X POST "https://demo.prescribe-rx.com/api/v1/bulk-orders" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "external_order_id": "PO-2026-0042",
  "shipping_address": { "street": "...", "city": "...", "state": "GA", "zip": "3
  "lines": [
    { "product_id": "...", "quantity": 50 },
    { "product_id": "...", "quantity": 25 }
```

```
]
```

Step 2 – poll for processing status

```
curl "https://demo.prescribe-rx.com/api/v1/bulk-orders/$REQUEST_ID/status" \
-H "Authorization: Bearer $TOKEN"
```

Step 3 – fetch the resulting order

Once processing completes, the response includes an `order_id`. Use the standard `GET /orders/{id}` for full details.

```
curl "https://demo.prescribe-rx.com/api/v1/orders/$ORDER_ID?include=fulfillment" \
-H "Authorization: Bearer $TOKEN"
```

4

Webhook subscribe + signature verify

When you're: replacing the status-poll loop in Flow 1 with real-time push

Webhooks fire on 27+ events (`encounter.prescribed` , `order.placed` , `fulfillment.shipped` , etc — full list at [webhooks guide](#)). Each delivery includes `X-PrescribeRx-Signature: sha256=<hex>` — verify with HMAC-SHA256 of the raw body using your subscription secret.

Step 1 – discover available event slugs

```
curl "https://demo.prescribe-rx.com/api/v1/webhooks/event-types" \
-H "Authorization: Bearer $TOKEN"
```

Step 2 – create a subscription

The response includes a one-time `secret` used to sign every delivery. Store it server-side; it's never returned again.

```
curl -X POST "https://demo.prescribe-rx.com/api/v1/webhooks" \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "name": "Prod integration",
  "url": "https://yourapp.example.com/webhooks/prescribe-rx",
  "events": ["encounter.prescribed", "order.placed", "fulfillment.shipped"],
  "is_active": true
}'
```

Step 3 – verify incoming deliveries

Every webhook delivery includes `X-PrescribeRx-Signature: sha256=<hex>`. Compute HMAC-SHA256 of the raw body bytes (not parsed JSON) keyed by your subscription secret and compare.

PHP

JavaScript

Python

```
function verifyWebhook(string $rawBody, string $signatureHeader, string $secret)
{
    if (! str_starts_with($signatureHeader, 'sha256=')) return false;
    $provided = substr($signatureHeader, 7);
    $expected = hash_hmac('sha256', $rawBody, $secret);
    return hash_equals($expected, $provided);
}
```

Test before going live. `POST /webhooks/{subscription}/test` triggers a synthetic delivery to your endpoint with the same signature shape as real events. `GET /webhooks/{subscription}/delivery-logs` shows per-attempt success/failure for production debugging.

5 Test-data flow with `is_sandbox`

When you're building / staging integration code without polluting production reports or paging on-call providers

Setting `"is_sandbox": true` on the unified intake (or letting the auto-detector do it via a "Test" -named patient) flags the encounter + any resulting order as sandbox data. The

flag cascades through the whole stack:

- ☒ Encounter created — provider auto-assignment **skipped** (capacity preserved)
- ☒ Order created — fulfillment dispatch **skipped** (no carriers / pharmacies hit)
- ☒ Excluded from commission + billing-statement reports
- ☒ Webhooks **still fire** (so you can test your handler end-to-end)

Explicit flag (recommended for CI / automated tests)

```
curl -X POST "https://demo.prescribe-rx.com/api/v1/telehealth/intake/unified" \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{
  "is_sandbox": true,
  "encounter_type_slug": "weight-management",
  "patient": { "first_name": "QA", "last_name": "Smoke", "email": "qa+'$(date +%Y-%m-%d)@example.com'"
}
```

Auto-detect via test-name regex

When the patient's first or last name matches `/\btest(ing|er)?\b/i`, `is_sandbox=true` is auto-applied. Useful for one-off manual tests where you want the same effect without remembering the flag.

```
# All of these auto-flag as sandbox:
"first_name": "Test"
"last_name": "Tester"
"first_name": "Testing"
"first_name": "John", "last_name": "Test"
```

Some tenants opt OUT of test-data via the org-level `block_test_intakes` setting — in that case auto-detection promotes to a 422 reject instead. Use the explicit `"is_sandbox": true` flag for production parity testing across all tenants.

What's next

These five cover the high-traffic flows. For everything else — patient self-service, vital history queries, prescription cancellation, lab orders, scheduling — see the full [interactive API reference](#) or the [field-mapping page](#) for what each payload field actually does on the server.